LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

TeamPCP Supply Chain Campaign: Activity Through 2026-06-07, (Mon, Jun 8th)

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-16
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

Published: 2026-06-16 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

TeamPCP Supply Chain Campaign: Activity Through 2026-06-07, (Mon, Jun 8th)

1. Executive Summary

Lost Boys Cyber assesses with medium confidence that the TeamPCP supply chain campaign remains operationally relevant to organizations that build, publish, or consume open-source developer tooling through npm, PyPI, GitHub Actions, and AI-assisted development environments. The June 8 SANS Internet Storm Center update reports two important developments since the prior 2026-05-24 activity summary: U.S. government reporting formally caught up to the campaign, and a broader attacker population is now able to reuse or adapt the Mini Shai-Hulud framework that TeamPCP reportedly open-sourced in May 2026.

This is not a single-product patch-management issue. The campaign centers on compromised build, package-publishing, and developer-workstation trust paths: stolen package tokens, GitHub Actions / OIDC abuse, trojanized package releases, post-install credential collection, and persistence or re-execution through developer tooling. Public reporting links the TeamPCP activity cluster to earlier compromises involving Aqua Trivy, Checkmarx KICS, LiteLLM, TanStack packages, AntV packages, SAP CAP-related npm packages, and other high-trust developer or AI/ML ecosystem packages. CVE-2026-45321 is publicly associated with at least part of the Mini Shai-Hulud / TanStack compromise activity, but defenders should treat the campaign as a moving supply-chain intrusion set rather than a CVE-bounded vulnerability.

Immediate priorities for defenders are to identify affected dependency paths, rotate tokens that could have been exposed through developer endpoints or CI/CD jobs, enforce provenance controls for package publishing, and hunt for unusual package-manager, GitHub, npm, PyPI, and AI-agent execution behavior. Manufacturing environments are particularly exposed when engineering, firmware, MES, OT-support, or product-security teams consume public packages or run security scanners and AI coding tools from build networks that can reach signing keys, cloud credentials, repositories, or deployment systems.

Field	Assessment
Report ID	LBC-VTI-2026-078
Intelligence focus	TeamPCP supply chain campaign through 2026-06-07
Primary risk	Credential theft and malicious package propagation through trusted developer tooling
Sector focus	Manufacturing and industrial technology organizations with CI/CD, firmware, or product-development pipelines
Confidence	Medium; SANS ISC is reliable, but full package/IOC scope changes quickly and should be validated against ecosystem advisories before enforcement
Action window	Immediate inventory, token rotation, and telemetry review for exposed build and developer systems

2. Intelligence Requirement

This report answers the following intelligence requirement: What does the latest TeamPCP / Mini Shai-Hulud supply chain activity mean for manufacturing-sector defenders, and what should security, engineering, and platform teams do next?

Priority intelligence questions:

PIR	Why It Matters	Current Answer
Which environments are exposed?	Supply-chain malware often lands first in developer workstations and CI/CD jobs, not	Any system that installs npm/PyPI packages, runs security scanners, uses

	production servers.	GitHub Actions, publishes packages, or executes AI coding-agent hooks should be considered in scope.
Is this only a TanStack CVE problem?	Narrow CVE handling can miss credential theft and propagation paths.	No. CVE-2026-45321 is one public anchor, but TeamPCP activity spans multiple package ecosystems and trusted tool chains.
What should be hunted first?	Logs are noisy and retention is limited.	Focus on package install scripts, token usage anomalies, new/modified GitHub Actions workflows, suspicious AI-tool hooks, and package publish events following developer or CI credential access.
What defensive changes reduce recurrence?	Rotating tokens once does not fix the trust model.	Enforce least-privilege tokens, short-lived publishing credentials, provenance/attestation, dependency pinning, maintainer MFA, and isolated build runners.

Assumptions used in this report:

- The organization consumes public open-source packages directly or indirectly through developer tooling, scanner containers, CI/CD workflows, or vendor-provided SDKs.
- The reader can collect endpoint, repository, package-manager, cloud IAM, and CI/CD telemetry for at least the last 30 days.
- Named package families and ecosystem examples are used as scoping pivots, not as a complete IOC list.

3. Source Review and Confidence

Source	Contribution	Reliability / Confidence	Notes
SANS ISC diary 33060, “TeamPCP Supply Chain Campaign: Activity Through 2026-06-07”	Primary trigger for this report; states the campaign evolved through U.S. government attention and wider reuse of Mini Shai-Hulud.	Reliable source, medium report confidence	RSS intake preserved only a summary excerpt in the local source bundle; full source should be checked before external publication if precise package lists are required.
SANS ISC prior TeamPCP tracking and “When the Security Scanner Became the Weapon” references	Campaign lineage: Trivy, Checkmarx KICS, LiteLLM, and security-tool supply-chain framing.	Medium-high	Corroborated by multiple security-vendor summaries surfaced in search results.
NVD / Tenable / Snyk public references for CVE-2026-45321	Anchor for TanStack / Mini Shai-Hulud CVE tracking.	Medium	The CVE should not be treated as the full campaign boundary.
Public vendor and media reporting from StepSecurity, Akamai, ReversingLabs, Dark Reading, Flashpoint, Snyk, and others	Package-ecosystem details, attacker propagation behavior, AI-tool persistence references, and affected ecosystem examples.	Medium	Useful for scoping and hunts; package lists change rapidly and must be validated against authoritative advisories.

Analytic confidence is medium. SANS ISC is a credible source, and the local intake record preserved the core claims. However, this report intentionally avoids over-claiming exact counts, package names, or infrastructure indicators where only

search snippets or secondary reporting were available during drafting. The defensive guidance is therefore framed around behaviors and exposed trust paths rather than a brittle IOC-only response.

4. Key Findings

Finding	Defensive Implication	Confidence
TeamPCP activity has moved beyond isolated package compromises into repeatable supply-chain tradecraft.	Response should include CI/CD, package publishing, developer endpoints, and cloud identity review, not only package upgrades.	Medium-high
Mini Shai-Hulud's public availability increases copycat and variant risk.	Organizations should hunt for behavior patterns even if package names do not match initial advisories.	Medium
The campaign abuses trusted tooling paths including security scanners, package managers, GitHub workflows, and AI-assisted development hooks.	Security tools and developer automations require the same provenance, isolation, and monitoring as production deployments.	Medium-high
Manufacturing organizations face elevated business risk when build pipelines connect software, firmware, cloud, and OT-support workflows.	Compromise of engineering CI/CD can create downstream product, customer, and plant-floor support exposure.	Medium
CVE-2026-45321 is a useful tracking artifact but not a complete remediation boundary.	Patch and package-specific remediation should be paired with credential rotation and supply-chain hardening.	Medium

Operational interpretation:

- Treat package installation and build execution as code execution from an untrusted source until provenance is verified.
- Treat any token accessible to a developer shell, CI runner, package post-install step, scanner, or AI coding agent as potentially exposed after suspected contact with compromised packages.
- Prioritize short-lived credentials and scoped publishing permissions over long-lived npm, PyPI, GitHub, cloud, or SaaS tokens.

5. Threat Context

TeamPCP is publicly described as an actor behind a multi-stage software supply-chain campaign that weaponized trusted developer and security tooling. Earlier reporting associated the campaign with compromises or abuse of Trivy, Checkmarx KICS, LiteLLM, and related ecosystem trust paths. Later Mini Shai-Hulud activity expanded the concern into self-propagating package compromise and credential theft across npm and PyPI-oriented development workflows.

The notable threat shift is reuse. If TeamPCP open-sourced or otherwise released the Mini Shai-Hulud framework, defenders should expect variant behavior from actors with different objectives: credential theft, package poisoning, extortion, repository manipulation, CI/CD persistence, or access brokering. This reduces the value of actor-name-only detection and increases the value of behavior-driven controls.

Campaign Element	Observed / Reported Pattern	Defensive Concern
Initial access	Compromised package releases, poisoned dependencies, or compromised maintainer/build credentials	Malicious code executes in developer or CI context during install/build
Credential access	Theft of npm/PyPI/GitHub/cloud tokens and environment secrets	Enables package publishing, repository access, lateral movement, or downstream compromise

Propagation	Reuse of stolen publishing paths and CI/CD identity	One compromised maintainer or runner can affect many downstream consumers
Persistence	Public reporting references GitHub workflow changes and AI-tool hook abuse in Mini Shai-Hulud variants	Developer tooling can re-execute payloads outside traditional malware persistence locations
Impact	Potential exposure of source code, CI secrets, deployment credentials, and customer-facing packages	Business impact can extend beyond IT into product integrity and customer trust

Manufacturing relevance is high when software development supports plant operations, industrial products, embedded firmware, connected devices, quality systems, or customer support tooling. A malicious dependency in an engineering pipeline may not immediately touch OT networks, but it can expose signing keys, update mechanisms, customer artifacts, and credentials used to bridge IT and operational support environments.

MITRE ATT&CK mapping:

Technique	Name	Evidence / Rationale
T1195.001	Supply Chain Compromise: Compromise Software Dependencies and Development Tools	Campaign centers on compromised packages and developer/security tooling.
T1552.001	Unsecured Credentials: Credentials In Files	Build and developer environments commonly expose secrets through environment files, CI variables, and local configuration.
T1552.007	Unsecured Credentials: Container and Cloud Instance Metadata API	Relevant where CI runners or scanner containers can reach metadata services or cloud workload identity.
T1059	Command and Scripting Interpreter	Package install scripts and build hooks execute shell, Node.js, Python, or Bun logic.
T1078	Valid Accounts	Stolen GitHub, npm, PyPI, or cloud tokens enable authenticated actions.
T1098.004	Account Manipulation: SSH Authorized Keys	Representative persistence concern for developer hosts; validate if endpoint telemetry shows key changes.
T1608.001	Stage Capabilities: Upload Malware	Malicious packages or modified workflows stage capability into trusted distribution channels.
T1105	Ingress Tool Transfer	Downloaded payloads, package scripts, and workflow artifacts can retrieve follow-on code.

6. Detection and Hunting

Detection should combine package-manager telemetry, endpoint process telemetry, repository audit logs, CI/CD audit logs, and cloud IAM events. IOC-only detection is insufficient because package names, payload filenames, and infrastructure change quickly.

Signal	Telemetry Source	Analytic Goal
npm, npx, pnpm, yarn, pip, bun, or python spawning shells/network utilities during install or build	EDR process telemetry; CI job logs	Identify malicious package install scripts and post-install credential collection.
New or modified GitHub Actions workflows after package install or unusual maintainer token use	GitHub audit logs; repository diffs	Detect propagation and persistence in repositories.
Package publish events from unusual runners, geolocations, accounts, or just after token access	npm/PyPI/GitHub package registry logs	Identify stolen token use and downstream package poisoning.
Reads of cloud, npm, PyPI, GitHub, SSH, or AI-tool credential files by package manager child processes	EDR file telemetry	Detect credential harvesting from developer systems.
AI coding tool hook/task modifications or unexpected startup commands	Endpoint file telemetry; developer config baselines	Detect Mini Shai-Hulud-style persistence or re-execution through developer tools.
GitHub Actions OIDC token requests from unexpected workflows or branches	Cloud IAM, GitHub audit, CI logs	Detect identity abuse from compromised workflows.

Sigma-style endpoint rule:

```
title: Suspicious Package Manager Child Process Credential Access
id: lbc-vti-2026-078-package-manager-credential-access
status: experimental
description: Detects package managers or build tools spawning shell/interpreter activity that touches
common credential locations. Tune for developer and CI baselines before alerting broadly.
author: Lost Boys Cyber
references:
  - https://isc.sans.edu/diary/rss/33060
logsource:
  category: process_creation
  product: windows
detection:
  selection_parent:
    ParentImage|endswith:
      - '\\npm.cmd'
      - '\\npx.cmd'
      - '\\pnpm.cmd'
      - '\\yarn.cmd'
      - '\\pip.exe'
      - '\\python.exe'
      - '\\node.exe'
      - '\\bun.exe'
  selection_child:
    Image|endswith:
      - '\\cmd.exe'
      - '\\powershell.exe'
      - '\\pwsh.exe'
      - '\\wscript.exe'
      - '\\cscript.exe'
      - '\\curl.exe'
      - '\\wget.exe'
      - '\\ssh.exe'
      - '\\git.exe'
  selection_cmd:
    CommandLine|contains:
      - '.npmrc'
      - 'pypirc'
```

```
      - 'id_rsa'
      - 'GITHUB_TOKEN'
      - 'ACTIONS_ID_TOKEN'
      - 'AWS_ACCESS_KEY'
      - 'AZURE_CLIENT_SECRET'
      - 'GOOGLE_APPLICATION_CREDENTIALS'
  condition: selection_parent and selection_child and selection_cmd
falsepositives:
  - legitimate build scripts that publish packages or run deployment tooling
  - developer bootstrap scripts in controlled repositories
level: high
```

Microsoft Defender XDR hunt for suspicious package-manager process chains:

```
let PackageParents = dynamic(["npm", "npm.cmd", "npx", "npx.cmd", "pnpm", "pnpm.cmd", "yarn",
"yarn.cmd", "pip", "pip.exe", "python", "python.exe", "node", "node.exe", "bun", "bun.exe"]);
let SuspiciousChildren = dynamic(["bash", "sh", "cmd.exe", "powershell.exe", "pwsh.exe", "curl",
"curl.exe", "wget", "wget.exe", "git", "git.exe", "ssh", "ssh.exe"]);
DeviceProcessEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ (PackageParents)
| where FileName in~ (SuspiciousChildren)
| where ProcessCommandLine has_any (".npmrc", "pypirc", "id_rsa", "GITHUB_TOKEN", "ACTIONS_ID_TOKEN",
"AWS_ACCESS_KEY", "AZURE_CLIENT_SECRET", "GOOGLE_APPLICATION_CREDENTIALS", "claude", "codex",
".github/workflows")
| project Timestamp, DeviceName, InitiatingProcessFileName, FileName, ProcessCommandLine,
InitiatingProcessCommandLine, AccountName, ReportId
| order by Timestamp desc
```

Microsoft Defender XDR hunt for developer credential file access following package installs:

```
let CredPaths = dynamic([".npmrc", ".pypirc", "id_rsa", "id_ed25519", "known_hosts", "credentials",
"config.json", "settings.json"]);
DeviceFileEvents
| where Timestamp > ago(30d)
| where FolderPath has_any (CredPaths)
| where InitiatingProcessFileName in~ ("npm", "npm.cmd", "npx", "npx.cmd", "pnpm", "pnpm.cmd", "yarn",
"yarn.cmd", "node", "node.exe", "python", "python.exe", "pip", "pip.exe", "bun", "bun.exe")
| project Timestamp, DeviceName, ActionType, FolderPath, FileName, InitiatingProcessFileName,
InitiatingProcessCommandLine, AccountName
| order by Timestamp desc
```

GitHub audit log hunt for workflow and package-publishing anomalies:

```
index=github sourcetype=github:audit
(action="repo.add_member" OR action="repo.config.disable_anonymous_git_access" OR action="workflows.*"
OR action="packages.*" OR action="oauth_authorization.*" OR action="org.update_member")
earliest=-30d
| eval suspicious=if(match(action,"workflow|package|oauth|token"),1,0)
| stats count values(action) as actions values(actor) as actors values(repo) as repos
values(user_agent) as user_agents by org, actor_ip
| where count > 3 OR mvcount(repos) > 5
| sort - count
```

Cloud IAM / OIDC hunt for unexpected CI token exchange:

```
// Adapt table names to the local SIEM connector. The goal is to find CI-issued identities used outside
expected repo, branch, or workflow context.
AuditLogs
| where TimeGenerated > ago(30d)
| where tostring(TargetResources) has_any ("GitHub", "Actions", "OIDC", "federated", "workload
identity")
| where OperationName has_any ("Add federated identity credential", "Update application", "Add service
principal", "Issue token", "AssumeRoleWithWebIdentity")
| project TimeGenerated, OperationName, InitiatedBy, TargetResources, Result, CorrelationId
| order by TimeGenerated desc
```

Hunt hypotheses:

- A developer or CI runner installed a compromised package and then accessed npm, PyPI, GitHub, SSH, cloud, or AI-tool credential material within the same session.
- A repository workflow was modified shortly after dependency installation, token creation, maintainer login, or package publish activity.
- Package publication occurred from a new network, automation runner, GitHub Actions workflow, or maintainer account pattern not present in the previous 90 days.
- AI coding-tool configuration files gained startup hooks, session hooks, tasks, or shell commands that re-run package manager payloads.

7. Recommendations

Priority	Owner	Action	Timeframe
Critical	Platform / DevOps	Inventory repositories, CI runners, package-publishing accounts, and developer groups that consumed affected ecosystem packages after 2026-03-19 and especially after 2026-05-01.	24-48 hours
Critical	IAM / Cloud	Rotate npm, PyPI, GitHub, cloud, artifact-registry, and deployment tokens exposed to developer workstations or CI jobs that may have installed compromised packages.	24-72 hours
High	Engineering	Pin dependencies, regenerate lockfiles from trusted sources, and review transitive packages connected to TanStack, AntV, SAP CAP, OpenSearch, Mistral AI, UiPath, Trivy, Checkmarx KICS, LiteLLM, and related advisories.	3-5 days
High	Security Operations	Deploy behavioral hunts for suspicious package-manager child processes, credential-file access, workflow changes, and anomalous package publishing.	3-5 days
High	DevSecOps	Enforce maintainer MFA, scoped publishing tokens, provenance/attestation, branch protections, protected package publishing, and pull-request review for workflow changes.	1-2 weeks
Medium	Endpoint / IT	Baseline developer AI-tool configuration and alert on new hooks/tasks that execute shell, package-manager, or network commands.	2 weeks
Medium	Manufacturing Security	Segment build and engineering	2-4 weeks

		environments from OT support credentials and production deployment secrets; review firmware/product signing key access.	
Medium	Legal / Risk	Prepare customer and supplier notification criteria for confirmed package poisoning or signing-key exposure.	As needed

Containment checklist for suspected exposure:

- Preserve CI job logs, repository audit logs, package registry logs, endpoint telemetry, and cloud IAM logs before retention windows expire.
- Disable or rotate long-lived package publishing tokens; replace with scoped, short-lived, workload-bound credentials.
- Review `.github/workflows/`, package release scripts, `postinstall` hooks, developer shell profiles, AI-tool hook directories, and package manager configuration for unexpected changes.
- Rebuild and republish trusted package versions from clean runners where compromise is suspected.
- Compare package tarballs/wheels against expected source commits and verified build artifacts.
- Treat downstream consumers as potentially affected if malicious versions were published, even if internal compromise is not yet proven.

8. References

- [1] SANS Internet Storm Center. “TeamPCP Supply Chain Campaign: Activity Through 2026-06-07, (Mon, Jun 8th).” https://isc.sans.edu/diary/rss/33060
- [2] SANS Internet Storm Center. TeamPCP campaign tracking and “When the Security Scanner Became the Weapon” references cited by the June 2026 diary.
- [3] Tenable. “Mini Shai-Hulud Supply Chain Attack CVE-2026-45321 FAQ.” https://www.tenable.com/blog/mini-shai-hulud-frequently-asked-questions
- [4] NVD. “CVE-2026-45321.” https://nvd.nist.gov/vuln/detail/CVE-2026-45321
- [5] Snyk. “TanStack npm Packages Hit by Mini Shai-Hulud.” https://snyk.io/blog/tanstack-npm-packages-compromised/
- [6] StepSecurity. “Mini Shai-Hulud Is Back: A Self-Spreading Supply-Chain Attack Hits the npm Ecosystem.” https://www.stepsecurity.io/blog/mini-shai-hulud-is-back-a-self-spreading-supply-chain-attack-hits-the-npm-ecosystem
- [7] Akamai Security Research. “Mini Shai-Hulud: The Worm Returns and Goes Public.” https://www.akamai.com/blog/security-research/mini-shai-hulud-worm-returns-goes-public
- [8] ReversingLabs. “Team PCP's Mini Shai-Hulud tears at open-source trust.” https://www.reversinglabs.com/blog/mini-shai-hulud-tears-at-oss-trust